Command-line app capstone

Module 6 - Lesson 3

Overview

A command-line application is a complete program you ship: it parses arguments, manages state, handles errors, and has tests. This capstone ties the course together.

Key Concepts

- Design the user interface: commands, flags, and help text.
- Organize code into modules: cli, core logic, and storage.
- Persist state (files, SQLite) so the app survives restarts.
- Handle errors gracefully and exit with meaningful status codes.
- Write tests for the core logic independent of the terminal.
- Document installation and usage in a README.

Example

```
import argparse

def main():
    parser = argparse.ArgumentParser(prog="tasks")
    sub = parser.add_subparsers(dest="command", required=True)
    add = sub.add_parser("add"); add.add_argument("title")
    ls = sub.add_parser("list")
    args = parser.parse_args()

    if args.command == "add":
        print(f"Added: {args.title}")
    elif args.command == "list":
        print("(no tasks)")

if __name__ == "__main__":
    main()
```

Summary

The capstone assembles argument parsing, modular code, persistence, and tests into a usable CLI app. It is a complete small product you can run, test, and share.

Practice Checklist

- Build a task tracker CLI with add, list, and complete commands.
- Persist tasks to a JSON file or SQLite.
- Add tests for the core task logic and a README.